DATABASE MANAGEMENT SYSTEMS
ASSIGNMENT 3 – CO3

Analysis, Normalisation and Validation of an Online Retail Order Management Database

Course: CSA05 – Database Management Systems

Assignment: 3 – CO3

Case Study: ShopNest Online Retail Order Management

Student Name: Preethi G

Register Number: 1924260136

Subject code;CSA0507

Important modelling assumptions

The answers follow the ShopNest scenario and attribute names in the supplied question paper. To make the mathematics precise, the main design assumes that a product occurs at most once in an order, so (OrderID, ProductID) identifies an order line. If the business permits the same product on multiple separate lines in one order, add LineNo and use (OrderID, LineNo) as the line key; the dependency analysis must then be adjusted.

The original table stores current product UnitPrice. The changed requirement in Task 12 is handled separately by introducing UnitPricePaid in ORDER_ITEM. TagList and ColourList are treated as independent sets of values for each product. For the 5NF example, the stated ternary supply rule is treated as a genuine three-way join dependency only when the business validates the pairwise-combination rule; it must not be assumed merely from sample data.

All sample data below is illustrative. SQL is executable design guidance. No live MySQL/PostgreSQL benchmark was run for this report; therefore Task 9 provides a reproducible measurement plan and expected qualitative trends, not fabricated measured values.

TASK 1 – Identify and Formalise the Functional Dependencies

The original relation is ORDER_LINE(OrderID, OrderDate, CustomerID, CustomerName, CustomerEmail, ShipStreet, ShipCity, ShipPIN, ShipState, ProductID, ProductName, UnitPrice, CategoryID, CategoryName, SupplierID, SupplierName, SupplierPhone, Quantity, WarehouseID, WarehouseCity, TagList, ColourList).

1.1 Functional dependencies

No.	Functional dependency	Business justification
1	CustomerID → CustomerName	A customer identifier identifies one customer name.
2	CustomerID → CustomerEmail	A customer identifier identifies one email address in the stated model.
3	OrderID → OrderDate	An order has one order date.
4	OrderID → CustomerID	An order belongs to one customer.
5	OrderID → ShipStreet, ShipCity, ShipPIN	An order is shipped to one address.
6	ShipPIN → ShipCity	A PIN code belongs to one city.
7	ShipPIN → ShipState	A PIN code belongs to one state.
8	(ShipStreet, ShipCity) → ShipPIN	A street in a city has one PIN code.
9	ProductID → ProductName	A product identifier identifies its name.
10	ProductID → UnitPrice	A product has one current price at a given point in time.
11	ProductID → CategoryID	A product belongs to one category.
12	CategoryID → CategoryName	A category identifier identifies its name.
13	ProductID → SupplierID	A product has one supplier in this scenario.
14	SupplierID → SupplierName, SupplierPhone	A supplier identifier identifies its name and phone.
15	WarehouseID → WarehouseCity	A warehouse identifier identifies its city.
16	(OrderID, ProductID) → Quantity	A particular product line in an order has one quantity.
17	(OrderID, ProductID) → WarehouseID	Each order line is dispatched from one warehouse.

The dependencies are non-trivial because the right-hand attributes are not contained in the left-hand determinant. Multi-attribute right sides can be split into single-attribute dependencies when calculating a canonical cover.

1.2 Multivalued dependencies

ProductID ↠ Tag and ProductID ↠ Colour hold when the set of tags attached to a product is independent of the set of colours offered for that product. For a product with tags {eco, cotton} and colours {red, blue}, all four tag-colour pairings are represented in the flattened three-attribute relation even though no business fact says that a particular tag belongs only to a particular colour.

1.3 Sample instance (12 tuples)

The following compact sample illustrates the line-level facts. Tag and colour sets are shown separately because the original TagList and ColourList are repeating groups; a fully flattened Tag–Colour relation would contain their Cartesian combinations.

OrderID	Date	CustID	Customer	Street	City/PIN	ProductID	Product	Qty	Warehouse
O101	2026-10-01	C01	Asha	12 Lake Rd	Chennai/600001	P01	Bottle	2	W01
O101	2026-10-01	C01	Asha	12 Lake Rd	Chennai/600001	P02	Notebook	3	W02
O102	2026-10-02	C02	Bala	8 Park St	Chennai/600002	P01	Bottle	1	W01
O102	2026-10-02	C02	Bala	8 Park St	Chennai/600002	P03	Pen	5	W02
O103	2026-10-03	C03	Chitra	4 Hill Rd	Madurai/625001	P02	Notebook	2	W02
O103	2026-10-03	C03	Chitra	4 Hill Rd	Madurai/625001	P04	Lamp	1	W03
O104	2026-10-04	C01	Asha	12 Lake Rd	Chennai/600001	P03	Pen	4	W02
O104	2026-10-04	C01	Asha	12 Lake Rd	Chennai/600001	P04	Lamp	1	W03
O105	2026-10-05	C02	Bala	8 Park St	Chennai/600002	P01	Bottle	2	W01
O105	2026-10-05	C02	Bala	8 Park St	Chennai/600002	P02	Notebook	1	W02
O106	2026-10-06	C03	Chitra	4 Hill Rd	Madurai/625001	P03	Pen	2	W02
O106	2026-10-06	C03	Chitra	4 Hill Rd	Madurai/625001	P04	Lamp	1	W03

The full 22-column tuple values are specified by the sample rows above; repeated values verify OrderID-level, CustomerID-level, ProductID-level, supplier, category, PIN and warehouse dependencies. For example, every occurrence of P01 has the same product name, current price, category and supplier; every occurrence of S01 has the same name and phone.

1.4 Candidate key and dependency diagram

Under the stated uniqueness assumption, the candidate key is (OrderID, ProductID). Neither OrderID nor ProductID alone determines the other or all line facts. Together they determine the complete line: OrderID determines order-level data, ProductID determines product-level data, and the pair determines Quantity and WarehouseID. The MVDs add independent repeating sets.

```
            (OrderID, ProductID)  [candidate key]
             /                  \
OrderID-level facts       ProductID-level facts
OrderDate                 ProductName, UnitPrice
CustomerID                CategoryID, SupplierID
ShipStreet/City/PIN       (then CategoryID → CategoryName)
(then PIN → City/State)   (then SupplierID → SupplierName/Phone)
```

```
                    \                    /
                     Quantity, WarehouseID
                     (WarehouseID → WarehouseCity)

Partial dependencies: OrderID → order/customer/shipping facts
                        ProductID → product/category/supplier facts
Full dependency: (OrderID, ProductID) → Quantity, WarehouseID
Transitive dependencies: ProductID → CategoryID → CategoryName;
                          ProductID → SupplierID → SupplierName/Phone;
                          OrderID → ShipPIN → ShipState/City.
```

The diagram is conceptual: ShipCity is also functionally determined by ShipPIN, and the street/city pair determines the PIN. The same attribute can participate in more than one valid dependency.

TASK 2 – Identify the Redundancy and Anomalies

The dominant problem is fact-level repetition, also called data redundancy. Redundancy is measured by how many extra copies of the same real-world fact are stored, not simply by the number of rows. One million rows may contain only a few thousand distinct products, customers and suppliers, so the same product and supplier facts are repeated many times.

2.1 Anomalies

- Update anomaly: Supplier S02 is stored with name Paper Co and phone 900000002 in several order lines. If the phone changes, every row containing S02 must be updated. Updating only some rows creates contradictory supplier records.
- Insertion anomaly: A new product P05 cannot be stored until it appears in an order if all product data is kept only in ORDER_LINE. A product with no order has no row in which its name, price, category and supplier can be recorded.
- Deletion anomaly: If the only order line for a product or the only order for a customer is deleted, the product/customer details may also disappear even though those entities still exist.

Example: deleting the last line containing P04 from O106 must not delete the independent facts that P04 is a Lamp supplied by S03. Likewise, changing S02's phone should require one update in SUPPLIER rather than changes to every order line.

2.2 Growth of redundancy

Order lines grow by about 20,000 per day, while the numbers of customers, products, suppliers, categories and warehouses grow much more slowly. Therefore the number of repeated copies grows roughly with order-line volume. If a popular product occurs in 40,000 order lines, its current product name, price, category and supplier may each be repeated 40,000 times in the flat table.

TASK 3 – Compute Closures, Keys and the Canonical Cover

3.1 Attribute closures

Closure algorithm: start with $X^+ = X$. Repeatedly add Y whenever a dependency LHS → Y has LHS contained in the current closure. Stop when no new attribute can be added.

```
(A) {OrderID}⁺
= {OrderID}
Add OrderDate, CustomerID, ShipStreet, ShipCity, ShipPIN.
From CustomerID add CustomerName, CustomerEmail.
From ShipPIN add ShipState (and ShipCity, already present).
Result: order date, customer and shipping facts; no product or line quantity.
```

```
(B) {ProductID}⁺
= {ProductID}
Add ProductName, UnitPrice, CategoryID, SupplierID.
From CategoryID add CategoryName.
From SupplierID add SupplierName, SupplierPhone.
Result: all product, category and supplier facts; no order or quantity.

(C) {OrderID, ProductID}⁺
Start with both attributes. Apply (A) and (B), then add Quantity and WarehouseID
from (OrderID, ProductID) → Quantity, WarehouseID. From WarehouseID add
WarehouseCity. This gives every attribute in ORDER_LINE (apart from separate
atomic Tag and Colour values, which are represented in their own relations).

(D) {CustomerID}⁺
= {CustomerID, CustomerName, CustomerEmail}. No order identifier is derived.

(E) {ShipPIN}⁺
= {ShipPIN, ShipCity, ShipState}. It does not determine ShipStreet.
```

3.2 Candidate key proof

OrderID and ProductID do not occur on the right side of any dependency in the stated FD set. Thus every key must contain both attributes. Their combined closure contains every single-valued attribute in the relation, so {OrderID, ProductID} is a superkey. Neither attribute can be removed: OrderID alone cannot determine ProductID or Quantity; ProductID alone cannot determine OrderID or Quantity. Therefore the pair is minimal and is the only candidate key under the modelling assumption.

Prime attributes: OrderID and ProductID. Non-prime attributes: all remaining attributes. Partial dependencies are those from OrderID alone or ProductID alone to non-prime attributes. Transitive dependencies include ProductID → CategoryID → CategoryName, ProductID → SupplierID → SupplierName/SupplierPhone, OrderID → ShipPIN → ShipState/ShipCity, and WarehouseID → WarehouseCity.

3.3 Canonical cover

Split every right-hand side into single attributes, then remove duplicate or implied dependencies. A compact minimal cover (grouped only for readability) is:

```
CustomerID → CustomerName
CustomerID → CustomerEmail
OrderID → OrderDate
OrderID → CustomerID
OrderID → ShipStreet
OrderID → ShipCity
OrderID → ShipPIN
ShipPIN → ShipCity
ShipPIN → ShipState
(ShipStreet, ShipCity) → ShipPIN
ProductID → ProductName
ProductID → UnitPrice
ProductID → CategoryID
CategoryID → CategoryName
ProductID → SupplierID
SupplierID → SupplierName
SupplierID → SupplierPhone
WarehouseID → WarehouseCity
(OrderID, ProductID) → Quantity
(OrderID, ProductID) → WarehouseID
```

No left-side attribute is extraneous in (ShipStreet, ShipCity) → ShipPIN: neither ShipStreet nor ShipCity alone is stated to identify the PIN. Neither OrderID nor ProductID is extraneous in the two line-level dependencies because each alone fails to determine Quantity or WarehouseID. The dependencies listed above are not

redundant under the stated business rules. If an actual data dictionary adds another rule, the cover should be recomputed.

Equivalence: each original FD is present directly or follows by decomposition/union from the cover. Conversely, every cover FD is in the original set or follows from its stated business rule. By Armstrong's reflexivity, augmentation and transitivity, the two sets imply one another; hence $F^+ = Fc^+$ for the single-valued FD set. MVDs are handled separately and are not part of this FD canonical cover.

TASK 4 – Analyse First and Second Normal Form

4.1 First Normal Form

1NF requires each cell to contain one atomic value and each row to be uniquely identifiable. TagList and ColourList contain collections, so they violate 1NF. Two common remedies are: (i) store one tag/colour per row in separate child relations PRODUCT_TAG(ProductID, Tag) and PRODUCT_COLOUR(ProductID, Colour), which preserves independent sets; or (ii) store a serialized list such as JSON in one column, which is convenient for retrieval but is not relationally atomic for ordinary joins, constraints and indexing. The first approach is preferred for a relational design.

4.2 Second Normal Form

2NF requires 1NF and no non-prime attribute to depend on a proper subset of a candidate key. Since the key is (OrderID, ProductID), OrderDate, CustomerID and shipping attributes depend only on OrderID; product name, current price, category and supplier attributes depend only on ProductID. These are partial dependencies.

4.3 2NF decomposition

Relation	Primary key	Important dependencies
ORDERS(OrderID, OrderDate, CustomerID, ShipStreet, ShipCity, ShipPIN)	OrderID	OrderID → order date, customer and shipping address
PRODUCT(ProductID, ProductName, UnitPrice, CategoryID, SupplierID)	ProductID	ProductID → product name, current price, category, supplier
ORDER_ITEM(OrderID, ProductID, Quantity, WarehouseID)	(OrderID, ProductID)	Pair → Quantity, WarehouseID
CUSTOMER(CustomerID, CustomerName, CustomerEmail)	CustomerID	CustomerID → name, email
CATEGORY(CategoryID, CategoryName)	CategoryID	CategoryID → CategoryName
SUPPLIER(SupplierID, SupplierName, SupplierPhone)	SupplierID	SupplierID → name, phone
WAREHOUSE(WarehouseID, WarehouseCity)	WarehouseID	WarehouseID → WarehouseCity
PRODUCT_TAG(ProductID, Tag)	(ProductID, Tag)	ProductID ↠ Tag
PRODUCT_COLOUR(ProductID, Colour)	(ProductID, Colour)	ProductID ↠ Colour

The decomposition removes partial dependencies from the line relation. It also reduces repeated customer, product and supplier facts. Insertion becomes possible without an order line; deleting an order line no longer deletes product or customer facts; and a supplier phone update is performed once in SUPPLIER. Some transitive dependencies remain inside ORDERS and PRODUCT and are removed in 3NF.

TASK 5 – Design the 3NF and BCNF Decompositions

5.1 3NF decomposition

Remove the transitive dependencies by separating determinants that are not keys of the containing relation:

Relation	Primary key	Purpose / dependencies
CUSTOMER(CustomerID, CustomerName, CustomerEmail)	CustomerID	Customer identity and contact facts
ORDERS(OrderID, OrderDate, CustomerID, ShipStreet, ShipCity, ShipPIN)	OrderID	Order facts; references CUSTOMER
POSTAL_ADDRESS(ShipStreet, ShipCity, ShipPIN)	(ShipStreet, ShipCity)	Street/city determines PIN
PIN_CODE(ShipPIN, ShipCity, ShipState)	ShipPIN	PIN determines city and state
PRODUCT(ProductID, ProductName, UnitPrice, CategoryID, SupplierID)	ProductID	Product facts
CATEGORY(CategoryID, CategoryName)	CategoryID	Category description
SUPPLIER(SupplierID, SupplierName, SupplierPhone)	SupplierID	Supplier facts
ORDER_ITEM(OrderID, ProductID, Quantity, WarehouseID)	(OrderID, ProductID)	Line facts
WAREHOUSE(WarehouseID, WarehouseCity)	WarehouseID	Warehouse facts
PRODUCT_TAG(ProductID, Tag)	(ProductID, Tag)	Product tags
PRODUCT_COLOUR(ProductID, Colour)	(ProductID, Colour)	Product colours

The order address can be represented by a separate address key in a production system; the above keeps the attributes close to the assignment's original names. Because ShipCity participates in both the address rule and PIN rule, a consistent implementation should use foreign keys or a canonical address table to prevent conflicting city values.

5.2 BCNF check

A relation is in BCNF if every non-trivial FD determinant is a superkey. CUSTOMER, CATEGORY, SUPPLIER, PIN_CODE, WAREHOUSE, PRODUCT_TAG and PRODUCT_COLOUR satisfy BCNF under their listed dependencies. PRODUCT satisfies BCNF if ProductID is its only determinant. ORDER_ITEM satisfies BCNF if (OrderID, ProductID) is its key and no additional line determinant exists. ORDERS satisfies BCNF for the order-level FDs after moving PIN-derived facts out, provided the remaining address FD is represented separately.

POSTAL_ADDRESS(ShipStreet, ShipCity, ShipPIN) has (ShipStreet, ShipCity) → ShipPIN, so that determinant is a key and the relation is BCNF. PIN_CODE has ShipPIN as key. If the schema instead keeps ShipPIN → ShipCity inside a relation whose key is (ShipStreet, ShipCity), that FD violates BCNF because ShipPIN is not a superkey; decomposing into POSTAL_ADDRESS and PIN_CODE resolves it.

5.3 Lossless join and dependency preservation

For a binary decomposition R → R1, R2, the standard lossless-join test is that the common attributes functionally determine all attributes of R1 or all attributes of R2 under the projected dependencies. For example, decomposing (ShipStreet, ShipCity, ShipPIN) into POSTAL_ADDRESS(ShipStreet, ShipCity, ShipPIN) and PIN_CODE(ShipPIN, ShipCity, ShipState) shares ShipPIN, and ShipPIN is the key of PIN_CODE; the join is lossless for consistent data. Each FD is enforced in the relation containing its determinant and dependent

attributes, so the decomposition is dependency-preserving for the stated FDs. Cross-table business rules may require foreign keys, UNIQUE constraints or triggers.

TASK 6 – Analyse Multivalued Dependencies and 4NF

An MVD X ↠ Y means that for each fixed X, the set of Y values is independent of the remaining attributes Z = R − X − Y. The tuple-swapping test says that if (x,y1,z1) and (x,y2,z2) occur, then (x,y1,z2) and (x,y2,z1) must also occur.

6.1 Tuple-swapping example

ProductID	Tag	Colour
P10	eco	red
P10	eco	blue
P10	reusable	red
P10	reusable	blue
P20	light	black
P20	light	white
P20	compact	black
P20	compact	white

For P10, eco/red and reusable/blue imply eco/blue and reusable/red, both present. The same test holds for P20. Thus ProductID ↠ Tag and ProductID ↠ Colour are supported by the sample.

6.2 Why BCNF is not enough; 4NF solution

PRODUCT_TAG_COLOUR(ProductID, Tag, Colour) can be in BCNF because its non-trivial FDs may have ProductID, Tag or Colour determinants absent; nevertheless it violates 4NF because ProductID ↠ Tag is a non-trivial MVD and ProductID is not a superkey of the three-column relation. A product with t tags and c colours creates t × c rows in the combined relation.

Decompose into PRODUCT_TAG(ProductID, Tag) and PRODUCT_COLOUR(ProductID, Colour). The same product now needs t + c rows rather than t × c rows. Example: 4 tags and 5 colours require 20 combinations in the combined relation but only 9 rows in the two decomposed relations. 4NF implies BCNF because every non-trivial FD is also an MVD; the converse is false because BCNF does not eliminate independent multivalued facts.

TASK 7 – Analyse Join Dependencies and 5NF

Let SUPPLY(SupplierID, CategoryID, WarehouseID) be a ternary relation. The proposed join dependency is *(SupplierID, CategoryID), (CategoryID, WarehouseID), (SupplierID, WarehouseID). It states that a triple is valid exactly when all three corresponding pairwise facts are present.

7.1 Why 4NF may not be sufficient

A relation can have no non-trivial MVD and therefore be in 4NF, yet still have a non-trivial join dependency involving three or more projections. Such a relation is not in 5NF if the join dependency is not implied by candidate keys. A decomposition into only two projections is not automatically lossless; it must pass the join test for the actual business relation.

Illustrative counterexample to an arbitrary two-way split: take a ternary relation with tuples (S1,C1,W1), (S1,C2,W2), (S2,C1,W2). Projecting to SC and SW and joining on SupplierID can create (S1,C1,W2) or

(S1,C2,W1) if those pair facts are independently present in the chosen instance. These spurious triples show why a two-projection decomposition must be tested, not assumed.

7.2 Three-way join dependency and decomposition

The 5NF decomposition is:

```
SUPPLIER_CATEGORY(SupplierID, CategoryID)
CATEGORY_WAREHOUSE(CategoryID, WarehouseID)
SUPPLIER_WAREHOUSE(SupplierID, WarehouseID)
```

To verify the JD, project the same SUPPLY instance into the three pair relations, join them, and compare the result with the original relation. The decomposition is lossless only if the join returns exactly the original triples. The assignment's business rule is the semantic justification; a sample alone cannot establish a universal constraint.

A reproducible six-or-more-tuple check should use a dataset for which the triple set equals the natural join of all three pair projections. The database test is: create the three pair tables, insert the projections, execute the three-way join, and compare both EXCEPT directions (or EXCEPT ALL where duplicates matter). If either direction returns rows, the JD is not satisfied.

```
SELECT sc.SupplierID, sc.CategoryID, cw.WarehouseID
FROM Supplier_Category sc
JOIN Category_Warehouse cw ON cw.CategoryID = sc.CategoryID
JOIN Supplier_Warehouse sw
  ON sw.SupplierID = sc.SupplierID
 AND sw.WarehouseID = cw.WarehouseID;

-- Compare this result with SUPPLY using EXCEPT in PostgreSQL,
-- or NOT EXISTS anti-joins in MySQL.
```

The tuple count before decomposition is |SUPPLY|. After decomposition it is |SC| + |CW| + |SW|; this may be larger or smaller depending on overlap, but each pair fact is stored once. 5NF is rarely enforced because decomposition increases joins and operational complexity. It is useful when a genuine cyclic business rule means that three pairwise relationships independently determine valid triples and updates must not create inconsistent combinations.

TASK 8 – Implement and Test the Schemas

The following PostgreSQL-style DDL gives the core BCNF/4NF implementation. MySQL can use equivalent InnoDB tables, CHECK constraints (supported in current versions), foreign keys and unique constraints.

```
CREATE TABLE customer (
  customer_id VARCHAR(12) PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL,
  customer_email VARCHAR(150) NOT NULL UNIQUE
);
CREATE TABLE supplier (
  supplier_id VARCHAR(12) PRIMARY KEY,
  supplier_name VARCHAR(100) NOT NULL,
  supplier_phone VARCHAR(20) NOT NULL
);
CREATE TABLE category (
  category_id VARCHAR(12) PRIMARY KEY,
  category_name VARCHAR(80) NOT NULL UNIQUE
);
CREATE TABLE warehouse (
  warehouse_id VARCHAR(12) PRIMARY KEY,
  warehouse_city VARCHAR(80) NOT NULL
);
CREATE TABLE pin_code (
```

```
  ship_pin VARCHAR(10) PRIMARY KEY,
  ship_city VARCHAR(80) NOT NULL,
  ship_state VARCHAR(80) NOT NULL
);
CREATE TABLE product (
  product_id VARCHAR(12) PRIMARY KEY,
  product_name VARCHAR(120) NOT NULL,
  current_unit_price NUMERIC(12,2) NOT NULL CHECK (current_unit_price >= 0),
  category_id VARCHAR(12) NOT NULL REFERENCES category(category_id),
  supplier_id VARCHAR(12) NOT NULL REFERENCES supplier(supplier_id)
);
CREATE TABLE orders (
  order_id VARCHAR(12) PRIMARY KEY,
  order_date DATE NOT NULL,
  customer_id VARCHAR(12) NOT NULL REFERENCES customer(customer_id),
  ship_street VARCHAR(150) NOT NULL,
  ship_pin VARCHAR(10) NOT NULL REFERENCES pin_code(ship_pin)
);
CREATE TABLE order_item (
  order_id VARCHAR(12) NOT NULL REFERENCES orders(order_id),
  product_id VARCHAR(12) NOT NULL REFERENCES product(product_id),
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  warehouse_id VARCHAR(12) NOT NULL REFERENCES warehouse(warehouse_id),
  PRIMARY KEY (order_id, product_id)
);
CREATE TABLE product_tag (
  product_id VARCHAR(12) NOT NULL REFERENCES product(product_id),
  tag VARCHAR(60) NOT NULL,
  PRIMARY KEY (product_id, tag)
);
CREATE TABLE product_colour (
  product_id VARCHAR(12) NOT NULL REFERENCES product(product_id),
  colour VARCHAR(40) NOT NULL,
  PRIMARY KEY (product_id, colour)
);
```

To enforce the address rule (ShipStreet, ShipCity) → ShipPIN, use an ADDRESS table with UNIQUE(ship_street, ship_city) and a stable address_id referenced by ORDERS. If the scenario requires a single PIN for a street within a city, this UNIQUE constraint enforces it. A product with no tags or colours is represented by zero child rows; it remains valid in PRODUCT. A product never ordered is also valid because PRODUCT does not depend on ORDER_ITEM.

8.1 Test cases

Test	Action	Expected result
FD violation	Insert same customer_id with a different customer_name	Rejected by primary key / update must affect one row
Unique email	Insert duplicate customer_email	Rejected by UNIQUE
Invalid order item	Insert unknown product_id or order_id	Rejected by foreign key
Quantity boundary	Insert quantity = 0	Rejected by CHECK
No tags	Insert product without PRODUCT_TAG rows	Accepted
Never ordered product	Insert PRODUCT with no ORDER_ITEM	Accepted
Delete only order of customer	Delete ORDERS row while order_item exists	Blocked unless child rows deleted / cascade explicitly chosen; customer remains
Supplier phone change	Update SUPPLIER once	One row updated; all joins show new phone
Independent tag/colour	Add one tag and one colour	One row added to each independent relation; no cross product stored

8.2 Loading and validation

Generate one deterministic source dataset and load equivalent logical data into the flat, 3NF/BCNF and 4NF/5NF schemas. Use fixed random seeds, identical products/customers/orders and the same query mix. Load parent tables before child tables. Record rejected inserts by test category. The flat schema should reproduce update, insertion and deletion anomalies; normalized tables should prevent entity-fact duplication by design.

Scale the dataset to 1,000, 10,000, 100,000 and 1,000,000 order lines. Use transactions and batch inserts. Add indexes on foreign keys and common query filters. The original flat design must keep all repeating groups atomic for a fair 1NF comparison; a comma-separated list is not a valid relational implementation.

TASK 9 – Record Experimental Evidence

No database engine was executed for this report, so exact byte sizes, elapsed times and rows-touched values cannot honestly be reported as measured results. The following is the required evidence table to complete after running the scripts on the same machine and database configuration.

Order lines	Flat table bytes	3NF/BCNF total bytes	4NF/5NF total bytes	Flat rows	Normalized total rows
1,000	Measure	Measure	Measure	1,000	Measure
10,000	Measure	Measure	Measure	10,000	Measure
100,000	Measure	Measure	Measure	100,000	Measure
1,000,000	Measure	Measure	Measure	1,000,000	Measure

Experiment	Measurement method	Expected qualitative result
Storage	PostgreSQL pg_total_relation_size / MySQL information_schema	Flat table repeats entity facts; normalized schemas add indexes and relation overhead but reduce duplicated text.
Supplier phone update	Time UPDATE and record affected rows	Flat: many order lines may be updated; normalized: one supplier row.
Order history	Run identical indexed query and capture EXPLAIN ANALYZE	Flat: fewer joins but wider rows; normalized: more joins, narrower tables and index lookups.
Constraint violations	Attempt invalid FK, duplicate key and CHECK values	Normalized design rejects violations at constraint boundaries.
Redundant copies	Count occurrences per distinct product/supplier/customer fact	Flat copy count tracks order-line frequency; normalized copy count is one per entity.

Example PostgreSQL measurement commands:

```
SELECT pg_total_relation_size('order_item') AS bytes;
EXPLAIN (ANALYZE, BUFFERS)
SELECT o.order_id, o.order_date, p.product_name, oi.quantity,
       oi.warehouse_id, s.supplier_name
FROM orders o
JOIN order_item oi ON oi.order_id = o.order_id
JOIN product p ON p.product_id = oi.product_id
JOIN supplier s ON s.supplier_id = p.supplier_id
WHERE o.customer_id = 'C01'
ORDER BY o.order_date DESC;
```

For MySQL, use EXPLAIN ANALYZE where supported and query information_schema.tables for table sizes. Repeat each benchmark several times, report median and spread, warm up the cache, and document hardware, DB version, indexes, buffer settings and transaction isolation.

TASK 10 – Compare Theory with Experimental Behaviour

Dependency theory predicts that the flat table repeats customer, product, category, supplier, warehouse and postal facts for every applicable order line. A normalized design stores each entity fact once and references it by key. After measurements, compare the actual duplicate counts with the number of occurrences predicted by the sample's frequency distribution.

- Updates: normalized relations should scale better for single-fact updates because one supplier/customer/product row is changed rather than every order line containing that fact.
- Order-history reads: the flat table may avoid joins, but it reads wider rows and carries redundant data. The normalized design needs joins; appropriate indexes and selective retrieval can make it efficient.
- Indexes: indexes increase storage and write cost but reduce lookup time. Foreign-key checks add insert/update work. Page size and buffer cache affect I/O. Popular products create skew: their rows and index pages may be hot, and the flat schema repeats their data more heavily.

Theory predicts direction, not exact milliseconds. The measured result depends on hardware, cache state, index design, data distribution and query selectivity. Report deviations rather than changing the theoretical explanation to fit one run.

TASK 11 – Recommend and Defend the Final Schema

The recommended operational design is a 3NF/BCNF core with 4NF separation for independent product tags and colours. Use 5NF decomposition for SUPPLY only if the cyclic three-way join dependency is confirmed as a real business constraint.

Relation	Recommended normal form	Reason
CUSTOMER, SUPPLIER, CATEGORY, WAREHOUSE, PIN_CODE	BCNF	Entity determinants are candidate keys; avoids repeated descriptions.
PRODUCT	BCNF	ProductID determines current product attributes and supplier/category references.
ORDERS	BCNF	OrderID determines order date, customer and shipping address reference.
ORDER_ITEM	BCNF	Composite key determines quantity and dispatch warehouse.
PRODUCT_TAG, PRODUCT_COLOUR	4NF / BCNF	Independent multivalued facts are separated.
SUPPLY pair relations	5NF if JD is validated	Avoids storing a ternary fact redundantly when it is exactly implied by three pair relationships.

The normalized design provides entity integrity, referential integrity, uniqueness and domain checks. Its costs are additional joins, more tables, more indexes and the need to maintain constraints. At 20,000 new order lines per day, this is a practical trade-off: product/customer/supplier changes remain small and consistent while order-item inserts are append-oriented. As order volume grows, partitioning ORDER_ITEM by order date, indexes and read replicas/materialized reporting views can support scale without duplicating authoritative facts.

TASK 12 – Re-evaluate and Adapt the Design

A. Impact analysis

The new requirement changes two things: customers must see the exact price paid at purchase time, and the system must answer 200,000 order-history queries per day. UnitPrice is no longer functionally determined by ProductID for historical order lines. ProductID → CurrentUnitPrice still holds in PRODUCT, but the historical

amount paid is a transaction fact determined by the order line: (OrderID, ProductID) → UnitPricePaid. Weekly price changes make it incorrect to reconstruct old receipts using today's PRODUCT price.

Query volume is important because a normalized order-history request joins ORDERS, ORDER_ITEM, PRODUCT and possibly SUPPLIER, WAREHOUSE and other tables. At 200,000 queries per day, latency, indexes, caching and read scaling matter. Historical facts should remain stable even when current master data changes.

B. Evaluation of original design

If the original design joins an old order line to PRODUCT and displays PRODUCT.UnitPrice, it may show the current price rather than the price charged on the order date. That is incorrect for receipts, refunds and customer disputes. The normalized design is still sound for master data, but it must be modified to preserve the transaction-time price.

A price-history table can answer what the catalogue price was on a date, but it may not represent discounts, coupons, negotiated prices or manual adjustments actually applied to a particular customer. Storing UnitPricePaid in ORDER_ITEM is therefore the authoritative solution. A covering index and read cache/materialized view can reduce query cost.

C. Adaptation / redesigned schema

Decision: MODIFY the original normalized design, not return to the flat table. Add UnitPricePaid (and, if required, DiscountAmount, TaxAmount and CurrencyCode) to ORDER_ITEM. It is not redundant duplication: it records the immutable commercial fact for that particular purchase, while PRODUCT.CurrentUnitPrice records the current catalogue price.

```
ALTER TABLE order_item
  ADD COLUMN unit_price_paid NUMERIC(12,2) NOT NULL
  CHECK (unit_price_paid >= 0);

CREATE INDEX idx_orders_customer_date
  ON orders(customer_id, order_date DESC);

CREATE INDEX idx_order_item_order
  ON order_item(order_id);

-- Optional covering index, subject to engine/version support:
CREATE INDEX idx_order_item_history
  ON order_item(order_id, product_id)
  INCLUDE (quantity, unit_price_paid, warehouse_id);
```

Revised dependency: (OrderID, ProductID) → UnitPricePaid, Quantity, WarehouseID. ProductID → CurrentUnitPrice remains valid for the current catalogue only. The historical price does not depend on ProductID alone because the same product can be sold at different prices in different orders and weeks.

Storage increases by one numeric value per order line. This is a deliberate and necessary cost. Query cost can be reduced by a composite index on customer/date and a covering index on order items. Integrity improves because the receipt price is preserved even after catalogue updates. The price-history relation may additionally be kept for audit and catalogue analysis, with effective_from/effective_to timestamps and a non-overlap constraint.

D. Evidence-based defence

The modified design follows from the dependency analysis: a product's current price and the price paid for an individual order line are different facts with different determinants. Normalization does not mean removing every repeated-looking value; it means storing each fact with the correct key. UnitPricePaid is repeated across purchases only when the same amount was genuinely paid on different lines, and each stored value belongs to a separate transaction event.

For 200,000 daily history queries, a normalized write model with indexed order/customer/date paths, a covering order-item index, caching and (if needed) a read replica is a balanced solution. A denormalized read model or materialized view may be added for performance, but it should be rebuildable from authoritative normalized tables. Weekly catalogue price changes then update PRODUCT without altering past receipts. This preserves integrity while addressing read performance.

Final recommendation: retain the 3NF/BCNF and 4NF decomposition, use the 5NF supply decomposition only when its join dependency is proven, and modify ORDER_ITEM to store UnitPricePaid. Measure the revised query mix before introducing further denormalization.

Conclusion

The original ORDER_LINE table combines order, customer, product, supplier, category, warehouse and postal facts, causing partial and transitive dependencies, redundancy and insertion, deletion and update anomalies. Decomposition into entity, order and order-item relations removes these problems. Separate tag and colour relations satisfy 4NF. A three-way supply decomposition is justified only when the join dependency is a genuine business rule. Under the changed requirement, storing UnitPricePaid in ORDER_ITEM preserves historical truth, while indexes and read-oriented optimizations support the high order-history query volume.

References / basis

Primary source: CSA05 – Database Management Systems, Assignment 3 – CO3, ShopNest scenario and Tasks 1–12 (uploaded assignment PDF, pages 1–7). The report uses standard relational database concepts: functional dependencies, attribute closure, Armstrong's axioms, candidate keys, canonical covers, lossless joins, dependency preservation, 1NF–5NF, multivalued dependencies and join dependencies.